

# E04 — Kth Largest Element in an Array (Heap / Quick Select)

Experiment: 4 | LeetCode: #215 | CO: CO3, CO5 | BT Level: BT5

---

## Problem Statement

Given an integer array `nums` and an integer `k`, return the **kth largest element** in the array.

Note that it is the kth largest element in sorted order, not the kth distinct element.

You must solve it in  $O(n)$  time complexity.

### Example 1:

Input: `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`

Output: 5

### Example 2:

Input: `nums = [3, 2, 3, 1, 2, 4, 5, 5, 6]`, `k = 4`

Output: 4

---

## Solution 1: Min-Heap of Size k

### Concept

Maintain a **min-heap of size k**. The heap's minimum is always the kth largest element seen so far.

- If a new element is larger than the heap's minimum → push it, pop the minimum.
- After processing all elements, heap's minimum is the answer.

```
import heapq
```

```
def findKthLargest_heap(nums, k):  
    """  
    Min-heap of size k.  
    Time:  $O(n \log k)$  | Space:  $O(k)$   
    """  
    min_heap = []  
  
    for num in nums:  
        heapq.heappush(min_heap, num)  
        if len(min_heap) > k:  
            heapq.heappop(min_heap) # Remove the smallest  
  
    return min_heap[0] # Top of min-heap = kth largest
```

## Heap Trace for `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`

| Num | Push | Heap      | Pop (if > k) | Heap after |
|-----|------|-----------|--------------|------------|
| 3   | 3    | [3]       | —            | [3]        |
| 2   | 2    | [2, 3]    | —            | [2, 3]     |
| 1   | 1    | [1, 3, 2] | pop 1        | [2, 3]     |
| 5   | 5    | [2, 3, 5] | pop 2        | [3, 5]     |
| 6   | 6    | [3, 5, 6] | pop 3        | [5, 6]     |
| 4   | 4    | [4, 6, 5] | pop 4        | [5, 6]     |

**Result:** `min_heap[0] = 5` ✓

---

## Solution 2: QuickSelect (Optimal $O(n)$ Average)

### Concept

QuickSelect is a partial quicksort: after partition, only recurse into the partition containing the *k*th element.

- We're looking for the **kth largest** = element at index  $(n - k)$  in a sorted array.
- Partition around pivot → if pivot is at index *p*:
  - If  $p == n - k \rightarrow$  found it!
  - If  $p < n - k \rightarrow$  search right half
  - If  $p > n - k \rightarrow$  search left half

```
import random

def findKthLargest_quickselect(nums, k):
    """
    QuickSelect (randomized).
    Time:  $O(n)$  average,  $O(n^2)$  worst | Space:  $O(1)$  in-place
    """
    target = len(nums) - k  # Index of kth largest in sorted order

    def partition(lo, hi):
        # Random pivot to avoid worst case
        pivot_idx = random.randint(lo, hi)
        nums[pivot_idx], nums[hi] = nums[hi], nums[pivot_idx]

        pivot = nums[hi]
        store = lo

        for j in range(lo, hi):
            if nums[j] <= pivot:
                nums[store], nums[j] = nums[j], nums[store]
                store += 1

        nums[store], nums[hi] = nums[hi], nums[store]
        return store
```

```

lo, hi = 0, len(nums) - 1
while lo < hi:
    pivot_pos = partition(lo, hi)
    if pivot_pos == target:
        break
    elif pivot_pos < target:
        lo = pivot_pos + 1
    else:
        hi = pivot_pos - 1

return nums[target]

```

## QuickSelect Trace for [3, 2, 1, 5, 6, 4], k = 2 → target index = 4

Array: [3, 2, 1, 5, 6, 4] lo=0, hi=5, target=4  
Pivot = nums[5] = 4

Partition:

j=0: 3 ≤ 4 → swap arr[0]↔arr[5]: [3, 2, 1, 5, 6, 4], store=1  
j=1: 2 ≤ 4 → swap arr[1]↔arr[5]: [3, 2, 1, 5, 6, 4], store=2  
j=2: 1 ≤ 4 → swap arr[2]↔arr[5]: [3, 2, 1, 5, 6, 4], store=3  
j=3: 5 > 4 → no swap  
j=4: 6 > 4 → no swap

Swap store↔hi: arr[3]↔arr[5]: [3, 2, 1, 4, 6, 5]  
Pivot position = 3

pivot\_pos=3 < target=4 → lo=4

Array: [3, 2, 1, 4, 6, 5] lo=4, hi=5, target=4  
Pivot = nums[5] = 5

Partition range [4,5]:

j=4: 6 > 5 → no swap, store=4

Swap store↔hi: arr[4]↔arr[5]: [3, 2, 1, 4, 5, 6]  
Pivot position = 4

pivot\_pos=4 == target=4 → return nums[4] = 5 ✓

---

## Complexity Comparison

| Solution     | Time                       | Space  | Best For                       |
|--------------|----------------------------|--------|--------------------------------|
| Min-Heap     | $O(n \log k)$              | $O(k)$ | Large n, small k; streaming    |
| QuickSelect  | $O(n)$ avg, $O(n^2)$ worst | $O(1)$ | Single-pass on in-memory array |
| Sort + Index | $O(n \log n)$              | $O(1)$ | Simple, not efficient          |

---

## Python Built-in

```
import heapq

def findKthLargest_builtin(nums, k):
    return heapq.nlargest(k, nums)[-1]    # heapq.nlargest returns k lar
```

---

## Test Suite

```
def test_kth_largest():
    test_cases = [
        ([3, 2, 1, 5, 6, 4], 2, 5),
        ([3, 2, 3, 1, 2, 4, 5, 5, 6], 4, 4),
        ([1], 1, 1),
        ([7, 6, 5, 4, 3, 2, 1], 5, 3),
        ([-1, -2, -3, -4, -5], 2, -2),
    ]
    for nums, k, expected in test_cases:
        r1 = findKthLargest_heap(nums[:], k)
        r2 = findKthLargest_quickselect(nums[:], k)
        for r, name in [(r1, "heap"), (r2, "quickselect")]:
            status = "PASS" if r == expected else "FAIL"
            print(f"{status} [{name}]: kth_largest({nums[:4]}..., {k})")

test_kth_largest()
```

---

## Key Takeaways

- **Min-heap of size k:** Maintains the k largest elements seen so far; top = kth largest. Space-efficient at  $O(k)$ .
  - **QuickSelect:** Average  $O(n)$  — optimal for this problem. Randomized pivot prevents worst-case.
  - Heap approach is ideal for **streaming data** (infinite stream, k-top queries).
  - QuickSelect is ideal for **static in-memory arrays**.
- 

## References

- LeetCode #215 — Kth Largest Element in an Array
- Cormen et al., *Introduction to Algorithms*, Ch. 9.2 (Selection in Expected Linear Time)
- Skiena, *The Algorithm Design Manual*, Ch. 4.3 (Priority Queues)